

启发式搜索

课堂实验（设计报告）

请各位同学使用A4纸进行提交：
1. 学号，姓名
2. 设计报告的内容

传教士和野人问题（The Missionaries and Cannibals Problem）：在河的左岸有 3 个传教士、3 个野人、1 条船。传教士想用这条船将所有成员都运送到河对岸，但是受到以下条件的约束：

a) 船每次最多只能装运两个；
b) 在任何岸边野人数目都不得超过传教士，否则传教士会遭遇危险。
c) 船每次不能空载

请基于课程中关于状态空间图与搜索方法，尝试规划出一种确保所有成员安全过河的计划。

解决问题思路包括：

1) 状态该如何表示？初始状态、结束状态分别是什么？状态空间有多少种，其中合理状态空间有哪些？
2) 动作集合包括什么？
3) 设计一种启发式算法（写出算法流程图）进行该问题求解，并说明该启发式算法是否是 A* 算法？

额外加分项目：

4) 假设有 M 个传教士，N 个野人，船每次最多只能装运 K 个。将上述启发式算法泛化到该问题求解。
5) 编写程序，输入 M、N 和 K，程序返回从初始状态到结束状态的运送方法。

第四章 对抗搜索

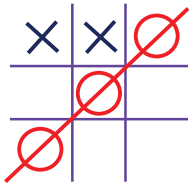
本章内容

- 4.1 博弈
- 4.2 博弈中的优化决策
- 4.3 α - β 剪枝
- 4.4 不完美的实时决策

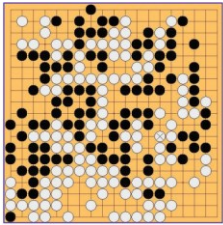
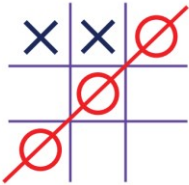
4.1 博弈

- 概述
- 博弈的类型
- 示例
- Grundy 博弈

概述



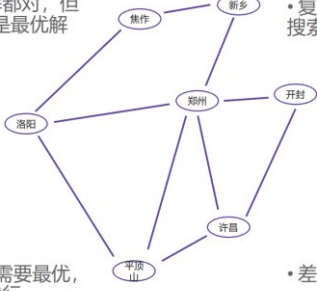
概述



- 搜索空间大，甚至几乎无限大（不完备）
- 环境会因自己的行为而改变（多Agent）
- 获胜的方式不止一种（最优解不唯一）

概述

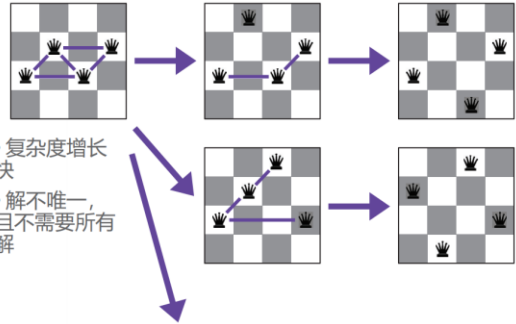
- 每个解都对，但难说它是最优解
- 复杂度增长快，搜索空间大
- 不一定需要最优，差不多就行
- 差不多是什么？



14

概述

- 复杂度增长快
- 解不唯一，且不需要所有解



15

概述

- 搜索空间极大，搜索天然不完备
- 存在多个最优解
- 或目标不求最优，“差不多”就行
- 搜索本身就是博弈
- 对抗
- 合作
- 对抗+合作



16

概述

- 在博弈问题中——比如说象棋——搜索是在博弈者双方之间进行的。
 - 任何一方在搜索时，都必须要考虑对方可能要采用的走步。
 - 对于一个优秀的博弈者来说，应考虑的不只是对方一步的走法，而是若干步的走法。
 - 这一过程一般来说是动态进行的。在考虑若干步走法以后，下了一步棋；而在对方走棋之后，还要再次考虑若干步走法，决定下一步的走法，而不是一劳永逸，搜索一次就决定了所有的走法。

17

概述

- 本章所讲的**博弈**：主要指的是类似于象棋这样的游戏问题。
- 这类问题有以下一些特点：
 - ① **双人对弈**，对垒的双方轮流走步。
 - ② **信息完备**，对垒双方得到的信息是一样的，不存在一方能看到，而另一方看不到的情况。
 - ③ **零和**。即对一方有利的棋，对另一方肯定是不利的，不存在对双方均有利、或均无利的棋。对弈的结果是一方赢，而另一方输，或者双方和棋。

18

概述

- **双人完备信息博弈**：
 - 指两位选手对垒，轮流走步，这时每一方不仅都知道对方过去已经走过的棋步，而且还能估计出对方未来可能的走步。
 - 对弈的结果是一方赢（另一方则输），或者双方和局。
 - 这类博弈的实例有：一字棋、余一棋、西洋跳棋、国际象棋、中国象棋、围棋等。
- **机遇性博弈**：存在不可预测性的博弈，例如掷币等。
 - 例如：西洋双陆棋。

19

4.1 博弈

• 博弈类型

- 确定还是随机?
 - 围棋, 象棋 VS 飞行棋, 大富翁
- 一个、两个或多个玩家?
- 零和游戏?
 - 围棋 VS 魂斗罗
- 全息游戏?
 - 围棋 VS DOTA



4.1 博弈

• 博弈类型

- 一般博弈
 - Agent的目标函数相互独立
 - 合作、竞争
- 零和博弈
 - Agent的目标函数对立
 - 最大化自己的目标函数和最小化敌人的效果相同
 - 纯粹的竞争



4.1 博弈

• 多玩家的确定性博弈

- S: 状态 (s_0 是起始状态)
- Player (s): 当前状态下, 采取行动的玩家。
- Actions (s): 当前状态下, 当前轮次玩家可以采取的动作集合。
- Results (s, a): 转移函数, 返回当前状态和动作的执行结果。
- Terminal Test (s): 终结测试, 是否为结束状态, 布尔值 {true, false}。
- Terminal Utilities (s, p): 效用函数, 判断结束状态的效用值。

4.1 博弈

• 海盗分金

- 假设现在有100枚金币
- 有A,B,C三个海盗来分
 - 海盗都是理性的, 只会做出最正确的选择
 - 海盗都是邪恶的, 不会相信其他人的承诺
- 要求:
 - A,B,C三人按顺序讲解自己的分配方案, 交由剩下人进行投票,
 - 如果超过 (大于等于) 半数通过, 则按照该方案进行分配,
 - 如果超过 (大于等于) 半数反对, 则将该人扔到海里喂鲨鱼,
- 提问: 最终的分配方案会是什么样的



4.1 博弈

• 示例: 非零和游戏

- 经典的博弈论问题: 囚徒困境
- 小明和小强联合犯罪, 被警方逮捕, 分别接受审讯。
- 小明和小强的可以选择沉默或者坦白, 并根据双方结果获得相应刑期。

		囚犯小明	
		沉默	坦白
囚犯小强	沉默	每人获得1个月刑期	小明直接释放 小强获得10个月刑期
	坦白	小明获得10个月刑期 小强直接释放	每人获得5个月刑期

4.1 博弈

• 囚徒困境问题升级版



4.1 博弈

- 示例2：智猪问题
- 猪圈里有一大一小两只智猪。猪圈一头是踏板，另一头是饲料出口
- 踩一下踏板，饲料出口就会出饲料
- 一只猪踩踏板，另一只猪就可以先吃到饲料
- 小猪踩踏板，大猪会在小猪跑到饲料出口之前吃光所有的食物
- 大猪踩踏板，则还有机会在小猪吃完落下的食物之前跑到饲料出口，吃到一半的食物
- 两只猪都踩踏板，则吃到的食物比是7：3

大猪 \ 小猪	踩	不踩
	7:3	5:5
踩	10:0	0:0

最终情况如何

4.1 博弈

- 智猪问题升级版（课堂练习）
- 猪圈里有一大一小两只智猪。猪圈一头是踏板，另一头是饲料出口
- 踩一下踏板，饲料出口就会出10份饲料
- 一只猪踩踏板，另一只猪就可以先吃到饲料，踩踏板的猪需要消耗2份食物才能跑过去吃到饲料
- 小猪踩踏板，大猪先吃，吃的比例是9:1
- 大猪踩踏板，小猪先吃，吃的比例是6:4
- 一起踩踏板，一起吃，吃的比例是7:3
- 都等着，都吃不到

最终情况如何

27

4.1 博弈

- 智猪问题升级版（课堂练习）
- 踩踏板的猪需要消耗2份食物才能跑过去吃到饲料
- 小猪踩踏板，大猪先吃，吃的比例是9:1
- 大猪踩踏板，小猪先吃，吃的比例是6:4
- 一起踩踏板，一起吃，吃的比例是7:3
- 都等着，都吃不到

大猪 \ 小猪	踩	不踩
	7:3	6:4
踩	9:1	0:0



28

4.1 博弈

- 智猪问题升级版（课堂练习）
- 踩踏板的猪需要消耗2份食物才能跑过去吃到饲料
- 小猪踩踏板，大猪先吃，吃的比例是9:1
- 大猪踩踏板，小猪先吃，吃的比例是6:4
- 一起踩踏板，一起吃，吃的比例是7:3
- 都等着，都吃不到

大猪 \ 小猪	踩	不踩
	5:1	4:4
踩	9:-1	0:0

纳什均衡点

29

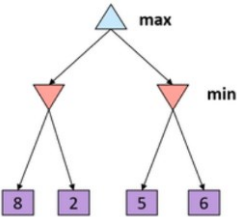
Grundy博弈

- Grundy博弈是一个分钱币的游戏。
- 有一堆数目为N的钱币，由两位选手轮流进行分堆，要求每个选手每次只把其中某一堆分成数目不等的两小堆。
- 例如，选手甲把N分成两堆后，轮到选手乙就可以挑其中一堆来分。
- 如此进行下去，直到有一位选手先无法把钱币再分成不相等的两堆时就就得认输。
- 对于这样的简单博弈问题，可以生成出它的状态空间图。这样就有可能从状态空间图中找出取胜的策略。

30

Grundy博弈

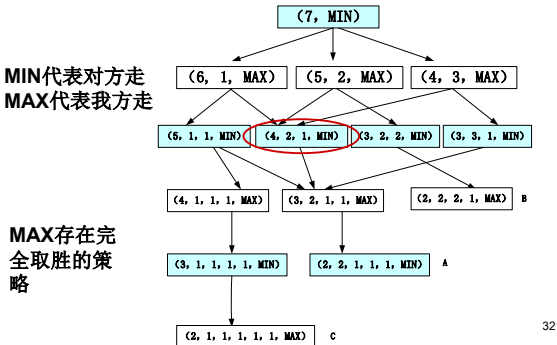
- 竞争的双方目标完全相反（零和）
- 常见于需要判胜负争高低的场景
- 象棋是零和吗？
- 世界杯是零和吗？
- 相声是零和吗？
- 相声比赛是零和吗？
- 高考是零和吗？
- 极大极小算法：在状态空间搜索树中，玩家交替操作，计算每个节点的最大和最小值



31

Grundy博弈

• 当初始钱币数为7时的状态空间图



Grundy博弈

- 搜索策略要考虑的问题：
 - 对MIN走步后的每一个MAX节点，必须证明MAX对MIN可能走的每一个棋局对弈后能获胜，即MAX必须考虑应付MIN的所有招法，这是一个与的含意。
 - 因此含有MAX符号的节点可看成与节点。
 - 对MAX走步后的每一个MIN节点，只须证明MAX有一步能走赢就可以，即MAX只要考虑能走出一歩棋使MIN无法招架就成。
 - 因此含有MIN符号的节点可看成或节点。
 - 因此，对弈过程的搜索图就呈现出与或图表示的形式。
- 33

Grundy博弈

- 寻找MAX的取胜策略和求与或图的解图相对应。
 - MAX要取胜，必须对所有与节点取胜，但只需对一个或节点取胜，这就是一个解图。
 - 因此，寻找一种取胜的策略就是搜索一个解图的问题，解图就代表一种完整的博弈策略。
 - 对于Grundy这种较简单的博弈，或者复杂博弈的残局，可以用类似于与或图的搜索技术求出解图，解图代表了从开局到终局任何阶段上的弈法。
 - 显然这对许多博弈问题是不能实现的。例如，中国象棋，国际象棋和围棋等。
- 34

Grundy博弈

- 对于复杂的博弈问题，因此即使用了强有力的启发式搜索技术，也不可能使分枝压到很少。
 - 因此，这种完全取胜策略（或和局）必须丢弃。
 - 应当把目标确定为寻找一步好棋，等对手回敬后再考虑寻找另一步好棋这种实际可行的实用策略。
 - 这种情况下每一步的结束条件可根据时间限制、存储空间限制或深度限制等因素加以确定。
 - 搜索策略可采用宽度、深度或启发式方法。一个阶段搜索结束后，要从搜索树中提取一个优先考虑的“最好的”走步，这就是实用策略的基本点。
- 35

本章内容

- 4.1 博弈
 - 4.2 博弈中的优化决策
 - 4.3 α - β 剪枝
 - 4.4 不完美的实时决策
- 36

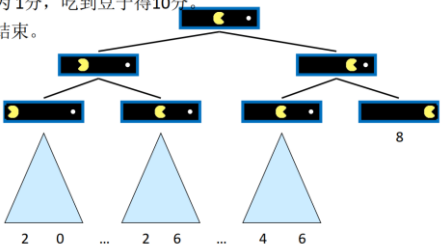
4.2 博弈中的优化决策

- 极小极大值法
 - 多人游戏中的最优决策（略）
- 37

对抗搜索树

• 单智能体搜索树

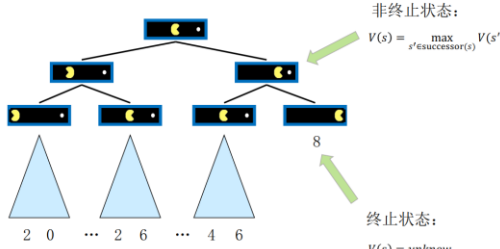
- 小黄人在长条形地图左右移动。
- 每次的动作损耗为 1分，吃到豆子得10分。
- 吃完豆子，游戏结束。



对抗搜索树

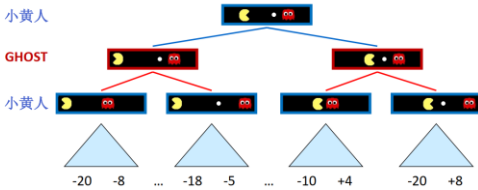
• 单智能体搜索树

- 状态的效用值：在当前状态下能得到的最大效用值



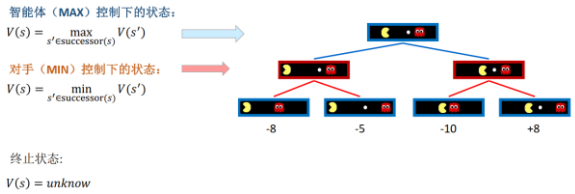
对抗搜索树

- 小黄人在长条形地图左右移动。
- GHOST在同样空间左右移动。
- 每次的动作损耗为 1分，吃到豆子得10分，触碰到鬼怪扣10分。
- 吃完豆子或者触碰到GHOST，游戏结束。

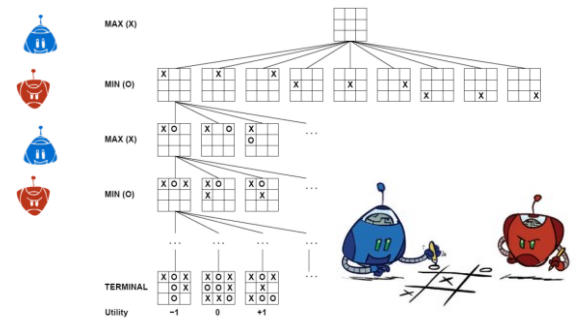


对抗搜索树

- 玩家都试图最大化自己的效用。
- 根据某玩家视角，主角最大化效用，对手最小化效用。



对抗搜索树



极小极大搜索过程

- 人类下棋的方法：实际上采用的是一种试探性的方法。
 - 首先假定走了一步棋，看对方会有那些应法，然后再根据对方的每一种应法，看我方是否有好的回应.....
 - 这一过程一直进行下去，直到若干步以后，找到了一个满意的走法为止。
 - 初学者可能只能看一、两个回合，而高手则可以看几个，甚至十几个回合。
- 极小极大搜索方法：模拟的就是人的这样一种思维过程。

极小极大搜索过程

- 极小极大搜索策略是考虑双方对弈若干步之后，从可能的走步中选一步相对好棋的着法来走，即在有限的搜索深度范围内进行求解。
- 定义一个静态估计函数 f ，以便对棋局的势态（节点）作出优劣估值。
 - 这个函数可根据势态优劣特征来定义（主要用于对端节点的“价值”进行度量）。
 - 一般规定：有利于MAX的势态， $f(p)$ 取正值；有利于MIN的势态， $f(p)$ 取负值；势均力敌的势态， $f(p)$ 取0值。
 - 若 $f(p) = +\infty$ ，则表示MAX赢，若 $f(p) = -\infty$ ，则表示MIN赢。

44

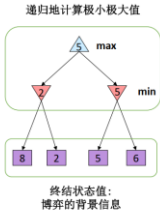
极小极大搜索过程

- 注意：不管设定的搜索深度是多少层，经过一次搜索以后，只决定了我方一步棋的走法。
 - 等到对方回应一步棋之后，需要新的棋局下重新进行搜索，来决定下一步棋如何走。
- 极小极大过程是一种假定对手每次回应都正确的情况下，如何从中找出对我方最有利的走步的搜索方法。

45

极小极大搜索过程

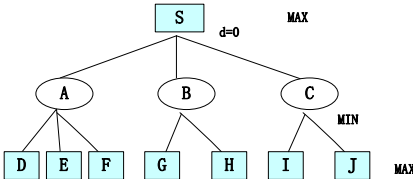
- 确定性的，零和游戏：
 - 井字棋，象棋
 - 一个玩家最大化游戏结果
 - 一个玩家最小化游戏结果
- 极小极大值搜索：
 - 构建状态空间搜索树
 - 两个玩家轮流进行操作
 - 计算每个节点的极小极大值：针对对手能获得的最佳效用值



46

极小极大搜索过程

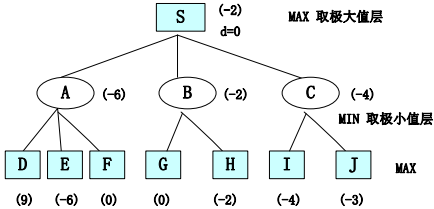
- 规定：顶节点深度 $d=0$ ，MAX代表程序方，MIN代表对手方。MAX先走。
- 例：一个考虑2步棋的例子。



47

极小极大搜索过程

- 规定：顶节点深度 $d=0$ ，MAX代表程序方，MIN代表对手方。MAX先走。
- 例：一个考虑2步棋的例子。



端节点给出的数字是用静态估计函数 $f(p)$ 计算得到。

48

过程MINIMAX

- $T := (s, MAX)$, $OPEN := (s)$, $CLOSED := ()$; 开始时树由初始节点构成， $OPEN$ 表只含有 s 。
- LOOP1: IF $OPEN = ()$ THEN GO LOOP2;
- $n := FIRST(OPEN)$, $REMOVE(n, OPEN)$, $ADD(n, CLOSED)$;
- IF n 可直接判定为赢、输或平局
THEN $f(n) := \infty \vee -\infty \vee 0$, GO LOOP1
ELSE EXPAND $(n) \rightarrow \{n_i\}$, $ADD(\{n_i\}, T)$
IF $d(n_i) < k$ THEN $ADD(\{n_i\}, OPEN)$, GO LOOP1
ELSE 计算 $f(n_i)$, n_i 达到深度 k , 计算各端节点 f 值。
- LOOP2: IF $CLOSED = NIL$ THEN GO LOOP3
ELSE $n_p := FIRST(CLOSED)$;
- IF $(n_p \in MAX) \wedge (f(n_p) \in MIN)$ 有值
THEN $f(n_p) := \max\{f(n_p)\}$, $REMOVE(n_p, CLOSED)$; 若MAX所有子节点均有值，则该MAX取其极大值。
IF $(n_p \in MIN) \wedge (f(n_p) \in MAX)$ 有值
THEN $f(n_p) := \min\{f(n_p)\}$, $REMOVE(n_p, CLOSED)$; 若MIN所有子节点均有值，则该MIN取其极小值。
- GO LOOP2;
- LOOP3: IF $f(s) \neq NIL$ THEN EXIT($END \vee M(Move, T)$); s 有值，则结束或标记走步。

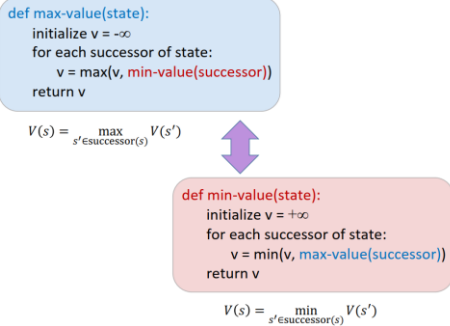
49

过程MINIMAX

- 该算法分两个阶段进行：
 - 第一阶段②~④：用宽度优先法生成规定深度的全部博弈树，然后对其所有端节点计算其静态估计函数值。
 - 第二阶段⑥~⑧：从底向上逐级求非端节点的倒推估计值，直到求出初始节点s的倒推值f(s)为止，此时
$$f(s) = \max_{i_1} \min_{i_2} \cdots \{f(n_{i_1 i_2 \cdots i_k})\}$$
其中 $n_{i_1 i_2 \cdots i_k}$ 表示深度为k的端结点。
- 等对手响应走步后，再以当前的状态作为起始状态s，重复调用该过程。

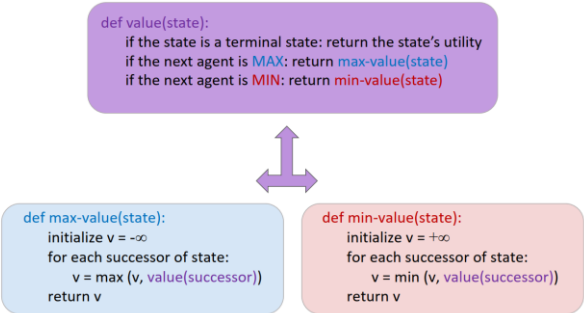
50

过程MINIMAX



51

过程MINIMAX



52

例：3×3棋盘的一字棋

- 3×3棋盘的一字棋：九宫格棋盘上，两位选手轮流在棋盘上摆各自的棋子（每次一枚），谁先取得三子一线的结果就取胜。
- 假设：
 - 程序方MAX的棋子用（×）表示；
 - 对手MIN的棋子用（○）表示；
 - MAX先走。

53

例：3×3棋盘的一字棋

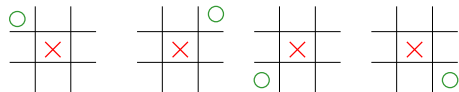
- 静态估计函数f(p)规定如下：
 - 若p是MAX获胜的格局，则f(p)=∞；
 - 若p是MIN获胜的格局，则f(p)=-∞；
 - 若p对任何一方来说都不是获胜的格局，则f(p)=(所有空格都放上MAX的棋子之后，MAX的三子成线（行、列、对角）的总数-（所有空格都放上MIN的棋子之后，MIN的三子成线（行、列、对角）的总数）。
- 例、当p的格局如下时，则可得f(p)=6-4=2。



54

例：3×3棋盘的一字棋

- 在搜索过程中，具有对称性的棋局认为是同一棋局，可以大大减少搜索空间。

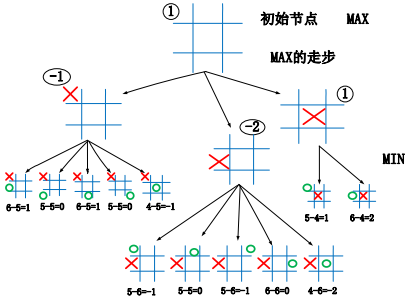


对称棋局的例子

55

例：3×3棋盘的一字棋

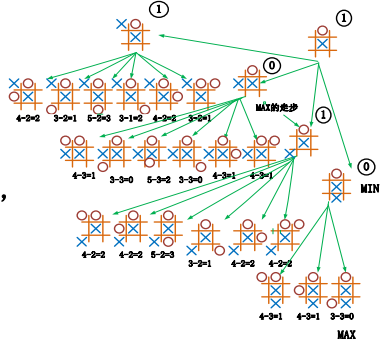
- 假定考虑走两步的搜索过程，利用棋盘对称性的条件，则第一次调用算法产生的搜索树为：



56

例：3×3棋盘的一字棋

- 假设MAX走完第一步后，MAX的对手是在X之上的格子下棋子，这时MAX在新格局下调用算法，第二次产生的搜索树为：

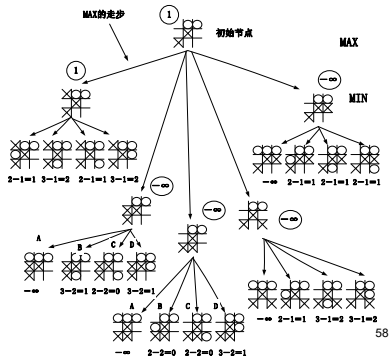


57

例：3×3棋盘的一字棋

- 类似地，第三次的搜索树为：

至此MAX走完最好的走步后，不论MIN怎么走，都无法挽回败局，因此只好认输了。

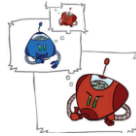


58

MINIMAX算法效率

- 极小极大的效率如何呢？
 - 类似（穷举的）深度优先算法
 - 时间复杂度： $O(b^m)$
 - 空间复杂度： $O(bm)$

b: 搜索分支因子
m: 最大深度

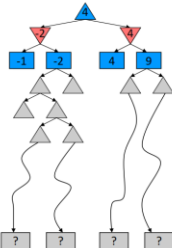


- 示例：国际象棋， $b \approx 35$, $m \approx 100$
 - 精确解法需要的计算量是不能承受的。
 - 对于一台每秒计算1,000,000 节点的计算机需要 8^{140} 年。
- 但是，我们需要探索整棵搜索树吗？

59

MINIMAX算法效率

- 问题：在真实的游戏中，无法搜索到叶子节点！
- 解决方案：截断搜索
 - 设定最大搜索深度
 - 针对非终止节点进行效用评估
- 无法保证最优性
 - 探索深度对结果的影响很大
- 如何确定合适的搜索深度？
 - 使用迭代加深



60

MINIMAX算法效率

- 在截断搜索中，评估函数对非终止节点进行效用值计算。
- 设计手工特征，对特征进行线性组合。

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

$$Eval(s) = material + mobility + king-safety + center-control$$
$$material = 10^{100}(K - K') + 9(Q - Q') + 5(R - R') + 3(B - B' + N - N') + 1(P - P')$$
$$mobility = 0.1(num\text{-}legal\text{-}moves - num\text{-}legal\text{-}moves')$$

- 基于表格的效用值评估
 - 在开局和收盘阶段可以发挥更大的效果。
- 基于机器学习的效用值评估

61

α - β 搜索算法

```

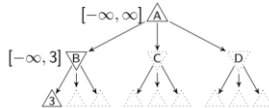
function ALPHA-BETA-SEARCH(state) returns an action
 $v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$ 
return the action in ACTIONS(state) with value v

function MAX-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
if TERMINAL-TEST(state) then return UTILITY(state)
for each c in ACTIONS(state) do
 $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(\text{a}, \alpha, \beta)))$ 
if  $v \geq \beta$  then return v
 $v \leftarrow \text{MAX}(v, v)$ 
return v

function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
if TERMINAL-TEST(state) then return UTILITY(state)
 $v \leftarrow -\infty$ 
for each c in ACTIONS(state) do
 $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(\text{a}, \alpha, \beta)))$ 
if  $v \leq \alpha$  then return v
 $v \leftarrow \text{MIN}(v, v)$ 
return v

```

- For MAX node
 - β is fixed as β_{parent}
 - v is used to update α
- For MIN node
 - α is fixed as α_{parent}
 - v is used to update β



74

α - β 搜索算法

```

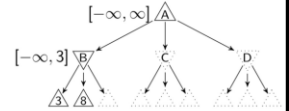
function ALPHA-BETA-SEARCH(state) returns an action
v ← MAX-VALUE(state, -∞, +∞)
return the action in ACTIONS(state) with value v

function MAX-VALUE(state, α, β) returns a utility value
if TERMINAL-TEST(state) then return UTILITY(state)
v ← -∞
for each a in ACTIONS(state) do
  v ← MAX(v, MIN-VALUE(RESULT(a, state), α, β))
  if v ≥ β then return v
return v ← MAX(v, v)

function MIN-VALUE(state, α, β) returns a utility value
if TERMINAL-TEST(state) then return UTILITY(state)
v ← +∞
for each a in ACTIONS(state) do
  v ← MIN(v, MAX-VALUE(RESULT(a, state), α, β))
  if v ≤ α then return v
return v ← MIN(v, v)

```

- For MAX node
 - β is fixed as β_{parent}
 - v is used to update α
- For MIN node
 - α is fixed as α_{parent}
 - v is used to update β



75

α - β 搜索算法

```

function ALPHA-BETA-Search(state) returns an action v
  v ← MAX-VALUE(state, −∞, +∞)
  return the action in ACTIONS(state) with value v

function MAX-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s, a),  $\alpha$ ,  $\beta$ ))
    if  $\beta \geq \alpha$  then return v
    v ← MAX(v, v)
  return v

function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← Min(v, MAX-VALUE(RESULT(s, a),  $\alpha$ ,  $\beta$ ))
    if v ≤ −∞ then return v
    v ← Min(v, v)
  return v

```

- For MAX node
 - β is fixed as β_{parent}
 - v is used to update α
- For MIN node
 - α is fixed as α_{parent}
 - v is used to update β



76

α - β 搜索算法

```

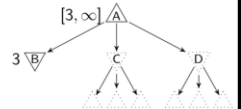
function ACTION-BETA-SEARCH( $d$ ) returns an action  $a$ 
 $v \leftarrow \text{MAX-VALUE}(d, -\infty, +\infty)$ 
return the action in ACTIONS( $d$ ) with value  $v$ 

function MAX-VALUE( $d, \alpha, \beta$ ) returns a utility value
if TERMINAL-TEST( $d$ ) then return UTILITY( $d$ )
for each  $a$  in ACTIONS( $d$ ) do
 $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(a, d), \alpha, \beta))$ 
if  $v \geq \beta$  then return  $v$ 
 $\alpha \leftarrow \text{MAX}(v, \alpha)$ 
return  $v$ 

function MIN-VALUE( $d, \alpha, \beta$ ) returns a utility value
if TERMINAL-TEST( $d$ ) then return UTILITY( $d$ )
for each  $a$  in ACTIONS( $d$ ) do
 $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(a, d), \alpha, \beta))$ 
if  $v \leq \alpha$  then return  $v$ 
 $\beta \leftarrow \text{MIN}(\beta, v)$ 
return  $v$ 

```

- For MAX node
 - β is fixed as β_{parent}
 - v is used to update α
- For MIN node
 - α is fixed as α_{parent}
 - v is used to update β



77

α - β 搜索算法

```

function ALPHA-BETA-SEARCH(state) returns an action
   $v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$ 
  return the action in ACTIONS(state) with value v

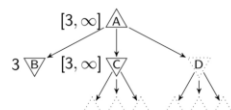
function MAX-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  for each a in ACTIONS(state) do
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$ 
    if  $v \geq \beta$  then return v
   $o \leftarrow \text{MAX}(O, v)$ 
  return o

function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  for each a in ACTIONS(state) do
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$ 
    if  $v \leq \alpha$  then return v
   $o \leftarrow \text{MIN}(O, v)$ 
  return o

```

For MAX node
 β is fixed as β_{parent}
 v is used to update α

For MIN node
 α is fixed as α_{parent}
 v is used to update β



78

α - β 搜索算法

```

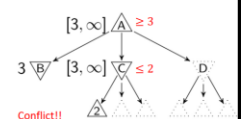
function MAX-ALPHA-BETA-SEARCH(state) returns an action a
   $\alpha \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$ 
  return the action in ACTIONS(state) with value  $\alpha$ 

function MAX-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  for each a in ACTIONS(state) do
     $v \leftarrow \text{MIN-VALUE}(\text{RESULT}(\text{state}, a), \alpha, \beta)$ 
    if  $v \geq \beta$  then return  $\alpha$ 
     $\alpha \leftarrow \text{MAX}(\alpha, v)$ 
  return  $\alpha$ 

function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow -\infty$ 
  for each a in ACTIONS(state) do
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(\text{state}, a), \alpha, \beta))$ 
    if  $v \leq \alpha$  then return  $v$ 
  return  $v$ 

```

- For MAX node
 - β is fixed as β_{parent}
 - v is used to update α
- For MIN node
 - α is fixed as α_{parent}
 - v is used to update β



79

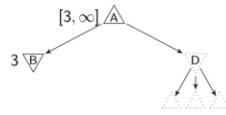
α - β 搜索算法

function ALPHA-BETA-SEARCH(*state*) returns an action
 $v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$
 return the action in ACTIONS(*state*) with value v

function MAX-VALUE(*state*, α , β) returns a utility value
 if TERMINAL-TEST(*state*) then return UTILITY(*state*)
 $v \leftarrow -\infty$
 for each a in ACTIONS(*state*) do
 $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(a, a), \alpha, \beta))$
 if $v \geq \beta$ then return v
 $\alpha \leftarrow \text{MAX}(\alpha, v)$
 return v

function MIN-VALUE(*state*, α , β) returns a utility value
 if TERMINAL-TEST(*state*) then return UTILITY(*state*)
 $v \leftarrow +\infty$
 for each a in ACTIONS(*state*) do
 $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(a, a), \alpha, \beta))$
 if $v \leq \alpha$ then return v
 $\beta \leftarrow \text{MIN}(\beta, v)$
 return v

For MAX node
 β is fixed as β_{parent}
 v is used to update α initialized as α_{parent}
 For MIN node
 α is fixed as α_{parent}
 v is used to update β initialized as β_{parent}



80

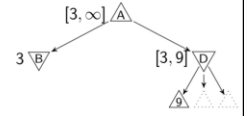
α - β 搜索算法

function ALPHA-BETA-SEARCH(*state*) returns an action
 $v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$
 return the action in ACTIONS(*state*) with value v

function MAX-VALUE(*state*, α , β) returns a utility value
 if TERMINAL-TEST(*state*) then return UTILITY(*state*)
 $v \leftarrow -\infty$
 for each a in ACTIONS(*state*) do
 $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(a, a), \alpha, \beta))$
 if $v \geq \beta$ then return v
 $\alpha \leftarrow \text{MAX}(\alpha, v)$
 return v

function MIN-VALUE(*state*, α , β) returns a utility value
 if TERMINAL-TEST(*state*) then return UTILITY(*state*)
 $v \leftarrow +\infty$
 for each a in ACTIONS(*state*) do
 $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(a, a), \alpha, \beta))$
 if $v \leq \alpha$ then return v
 $\beta \leftarrow \text{MIN}(\beta, v)$
 return v

For MAX node
 β is fixed as β_{parent}
 v is used to update α
 For MIN node
 α is fixed as α_{parent}
 v is used to update β



81

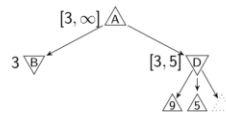
α - β 搜索算法

function ALPHA-BETA-SEARCH(*state*) returns an action
 $v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$
 return the action in ACTIONS(*state*) with value v

function MAX-VALUE(*state*, α , β) returns a utility value
 if TERMINAL-TEST(*state*) then return UTILITY(*state*)
 $v \leftarrow -\infty$
 for each a in ACTIONS(*state*) do
 $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(a, a), \alpha, \beta))$
 if $v \geq \beta$ then return v
 $\alpha \leftarrow \text{MAX}(\alpha, v)$
 return v

function MIN-VALUE(*state*, α , β) returns a utility value
 if TERMINAL-TEST(*state*) then return UTILITY(*state*)
 $v \leftarrow +\infty$
 for each a in ACTIONS(*state*) do
 $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(a, a), \alpha, \beta))$
 if $v \leq \alpha$ then return v
 $\beta \leftarrow \text{MIN}(\beta, v)$
 return v

For MAX node
 β is fixed as β_{parent}
 v is used to update α initialized as α_{parent}
 For MIN node
 α is fixed as α_{parent}
 v is used to update β initialized as β_{parent}



82

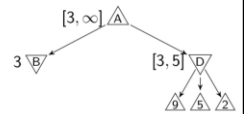
α - β 搜索算法

function ALPHA-BETA-SEARCH(*state*) returns an action
 $v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$
 return the action in ACTIONS(*state*) with value v

function MAX-VALUE(*state*, α , β) returns a utility value
 if TERMINAL-TEST(*state*) then return UTILITY(*state*)
 $v \leftarrow -\infty$
 for each a in ACTIONS(*state*) do
 $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(a, a), \alpha, \beta))$
 if $v \geq \beta$ then return v
 $\alpha \leftarrow \text{MAX}(\alpha, v)$
 return v

function MIN-VALUE(*state*, α , β) returns a utility value
 if TERMINAL-TEST(*state*) then return UTILITY(*state*)
 $v \leftarrow +\infty$
 for each a in ACTIONS(*state*) do
 $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(a, a), \alpha, \beta))$
 if $v \leq \alpha$ then return v
 $\beta \leftarrow \text{MIN}(\beta, v)$
 return v

For MAX node
 β is fixed as β_{parent}
 v is used to update α initialized as α_{parent}
 For MIN node
 α is fixed as α_{parent}
 v is used to update β initialized as β_{parent}



83

α - β 搜索算法

function ALPHA-BETA-SEARCH(*state*) returns an action
 $v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$
 return the action in ACTIONS(*state*) with value v

function MAX-VALUE(*state*, α , β) returns a utility value
 if TERMINAL-TEST(*state*) then return UTILITY(*state*)
 $v \leftarrow -\infty$
 for each a in ACTIONS(*state*) do
 $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(a, a), \alpha, \beta))$
 if $v \geq \beta$ then return v
 $\alpha \leftarrow \text{MAX}(\alpha, v)$
 return v

function MIN-VALUE(*state*, α , β) returns a utility value
 if TERMINAL-TEST(*state*) then return UTILITY(*state*)
 $v \leftarrow +\infty$
 for each a in ACTIONS(*state*) do
 $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(a, a), \alpha, \beta))$
 if $v \leq \alpha$ then return v
 $\beta \leftarrow \text{MIN}(\beta, v)$
 return v

For MAX node
 β is fixed as β_{parent}
 v is used to update α initialized as α_{parent}
 For MIN node
 α is fixed as α_{parent}
 v is used to update β initialized as β_{parent}



84

α - β 剪枝性质

- 剪枝不影响根节点的极大极小值的计算
- 中间节点的极大极小值在执行剪枝算法之后可能是错误的

- 遍历孩子节点的顺序对算法的执行有影响

- 好的遍历顺序可以提高剪枝效率
- 效用值递减遍历对 MAX 节点更有效
- 效用值递增遍历对 MIN 节点更有效



85

剪枝的效率问题

- 若以**最理想的情况**进行搜索，即对MIN节点先扩展最低估值的节点（若从左向右顺序进行，则设节点估计值从左向右递增排序），MAX先扩展最高估值的节点（设估计值从左向右递减排序），则当搜索树深度为D，分枝因数为B时，
 - 若不使用α-β剪枝技术，搜索树的端节点数为：

$$N_D = B^D$$

- 若使用α-β剪枝技术，可以证明理想条件下生成的端节点数最少，有

$$N_D = 2B^{D/2} - 1 \text{ (D为偶数)}$$

$$N_D = B^{(D+1)/2} + B^{(D-1)/2} - 1 \text{ (D为奇数)}$$

86

剪枝的效率问题

- 比较后得出**最佳α-β搜索技术**所生成深度为D处的端节点数约等于不用α-β搜索技术所生成**深度为D / 2处的端节点数**。
 - 这就是说，在一般条件下使用α-β搜索技术，在同样的资源限制下，可以向前考虑更多的走步数，这样选取当前的最好优先走步，将带来更大的取胜优势。

87

其他改进方法

- 使用α-β剪枝技术，当不满足剪枝条件（即α不大于β）时，若**β值比α值大不了多少或极相近**，这时也可以进行剪枝。
 - 以便有条件把搜索集中到会带来更大效果的其他路径上，这就是中止对效益不大的一些子树的搜索，以提高搜索效率。
- 不严格限制搜索的深度，当到达深度限制时，如出现**博弈格局有可能发生较大变化时**（如出现兑子格局），则应多搜索几层，使格局进入较稳定状态后再中止，这样可使倒推值计算的结果比较合理，避免考虑不充分产生的影响，这是等候状态平稳后中止搜索的方法。

88

其他改进方法

- 当算法给出所选的走步后，不马上停止搜索，而是在**原先估计可能的路径上再往前搜索几步**，再次检验会不会出现意外，这是一种增添**辅助搜索**的方法。
- 对某些博弈的**开局阶段**和**残局阶段**，往往总结有一些固定的对弈模式。
 - 因此，可以利用这些知识编好走步表，以便在开局和结局时使用**查表法**。只是在进入中盘阶段后，再调用其他有效的搜索算法，来选择最优的走步。

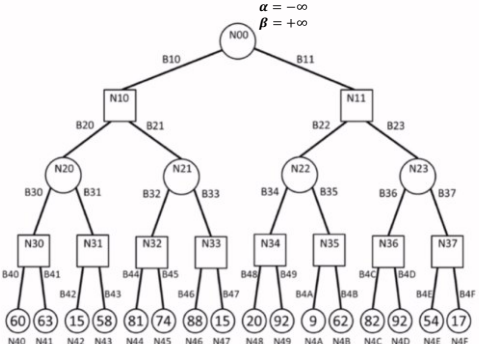
89

其他改进方法

- 以上这些方法还**不能**全面反映人们弈棋过程实际所使用的一切推理技术，也未涉及棋局的表示和启发函数问题。
 - 高明的棋手对棋局的表示有**独特的模式**。
- 博弈过程中，若在一个短时期内**短兵相接**，进攻和防御的战术变化剧烈，这些情况如何在搜索策略中加以考虑？
- 基于极小极大过程的一些方法都**设想对手走的总是最优走步**，即我方总应考虑最坏的情况。实际上，再好的选手也会有失误，如何利用失误加强攻势，也值得考虑。
- 总之要真正解决具体的博弈搜索技术，有许多更深入的问题需要作进一步的研究和探讨。

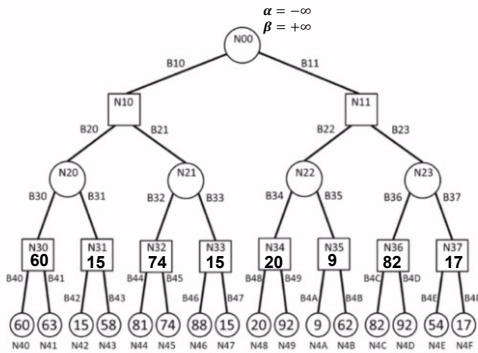
90

α-β搜索算法



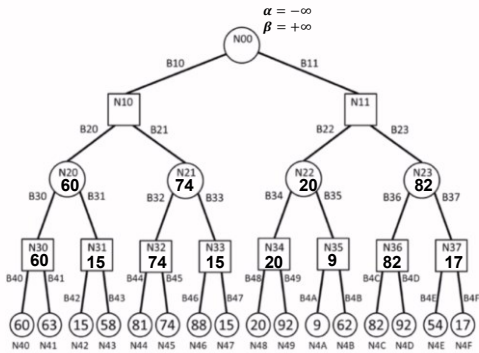
91

α-β搜索算法



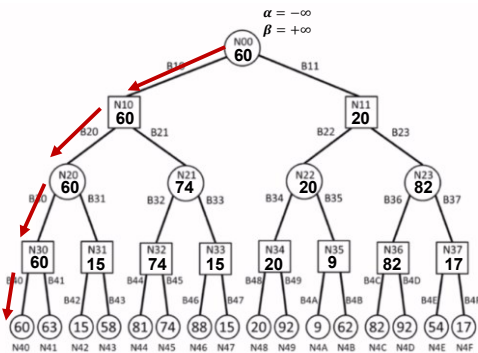
92

α-β搜索算法



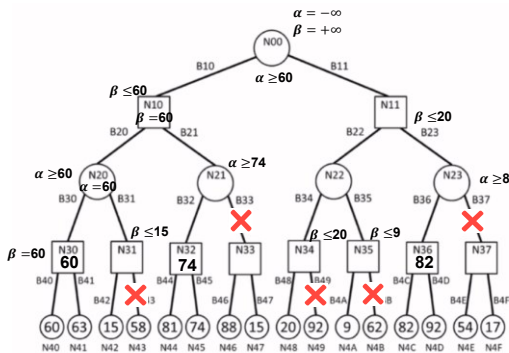
93

α-β搜索算法



94

α-β搜索算法



95

本章内容

- 4.1 博弈
- 4.2 博弈中的优化决策
- 4.3 α-β剪枝
- 4.4 不完美的实时决策

96

不完美的实时决策

- MINIMAX或α-β剪枝算法
 - 理想情况：算法一直搜索，直到至少一部分空间到达终止状态，从而对端节点做出准确评价。
 - 这样的搜索不现实。
- 实用方法：
 - 用可以估计棋局效用的启发式评价函数评价非终止节点。
 - 用可以决策什么时候运用评价函数的截断测试取代终止测试。

97

评价函数

- 评价函数的设计：
 - ① 应该以和真正的效用函数同样的方式对终止状态进行排序。
 - 效用函数（又称目标函数或者收益函数）：对终止状态给出一个数值。例如，在国际象棋中，结果是赢、输或平局，分别赋予+1，-1或0。
 - ② 评价函数的计算不能花费太多的时间！
 - ③ 对于非终止状态，评价函数应该和取胜的实际机会密切相关。

98

评价函数

- 在计算能力有限情况下，评价函数能做到最好的就是猜测最后的结果。
 - 例如，国际象棋并不是几率游戏，而且也确切知道当前状态；但计算能力有限，从而导致结果必然是不确定的。
- 大多数评价函数的工作方式是计算状态的不同特征。
 - 例如，国际象棋中兵的数目、象的数目、马的数目等等。
 - 这些特征一起定义了状态的各种类别或者等价类。
 - 但因类别太多而几乎不可能去估计取胜概率。

99

评价函数

- 大多数评价函数计算每个特征单独的数值贡献，然后把它们结合起来找到一个总值。
 - 加权线性评价函数
- $$EVAL(s) = w_1f_1(s) + w_2f_2(s) + \dots + w_nf_n(s) = \sum_{i=1}^n w_if_i(s)$$
- 每个 w_i 是一个权值， f_i 是棋局的某个特征。

100

评价函数

- 例如，国际象棋的入门书中给出各个棋子的估计子力价值。
 - 兵值1分；
 - 马、象值3分；
 - 车值5分；
 - 后值9分。
 - 其它特征。例如，“好的兵阵”和“王的安全性”可能值半个兵。
- 把这项特征值简单相加就得到了一个对棋局的估计。
- 经验表明，如果其它都一样，则
 - 在领先超过1分的可靠子力优势下很可能取得胜利；
 - 3分的优势几乎足以肯定取胜。

101

评价函数

- $$EVAL(s) = \sum_{i=1}^n w_if_i(s)$$
- 加权线性评价函数的假设：每个特征的贡献独立于其它特征的值。
 - 假设太强！
 - 例如，象赋予3分忽略了象在残局中能够发挥更大作用的事实。
 - 当前国际象棋和其它游戏的程序也采用非线性的特征组合。

103

评价函数

- $$EVAL(s) = \sum_{i=1}^n w_if_i(s)$$
- 注意：特征和权值并不是国际象棋规则的一部分。
 - 它们只是人类下棋的经验总结。
 - 在很难归纳这样的经验规律的游戏，怎么办？
 - 评价函数的权值可以通过机器学习来估计。

104

截断搜索

- 最直接的控制搜索次数的方法：**设置一个固定的深度限制**。
- 更鲁棒的方法：使用**迭代深入搜索**。
 - **具体实现**：不断加大深度优先限制。首先为0，接着为1，然后为2，依此类推。
 - 当时间用完时，程序就返回目前已完成的最深的搜索所选择的招数。

105

截断搜索

- 由于评价函数的近似性，这些方法可能导致错误。
 - 需要更为复杂的**截断测试**。
- 评价函数应该只用于那些**静止的棋局**。
 - **静止的棋局**：评价值在很近的未来不会出现大的摇摆变化棋局。
 - 例如，在国际象棋中，有很好的吃招的棋局对于只统计子力的评价函数来说就不能算静止的。
- **非静止的棋局**可以进一步扩展直到静止的棋局，这种额外的搜索称为**静止搜索**。
 - 有时候只考虑某些类型的招数，诸如吃子，能够快速解决棋局的不确定性。

106

单一扩展和前向剪枝

- **单一扩展**：搜索在给定的棋局中**一步**明显好于其它招数的行棋。
 - 对“**一步明显好于其它招数的行棋**”进行超过一般深度限制的搜索。
 - **目的**：避免地平线效应且不增加太多搜索代价。
- **前向剪枝**：在某个结点上不需要进一步搜索而直接剪枝。
 - 有危险！只在某些特殊的情况下使用才是安全的。

107

不完整的实时决策

- 假设已经实现：
 - 国际象棋的评价函数
 - 使用静止搜索的合理截断测试
 - 一个很大的调换表（存储以前见过的棋局的哈希表一般被称做**调换表**）
- 若在“最新的PC”上可每秒生成和评价约一百万个节点，则允许我们在标准的时间控制下（每步棋3分钟）对每步棋可搜索约2亿个节点。
 - 国际象棋的分支因子平均为35，35⁵约为5亿。
 - 如果使用极大极小搜索，只能向前预测**5层**，很容易被平均水平的人类棋手欺骗。
 - 如果使用alpha-beta搜索，可以预测大约**10层**，接近专业棋手水平。

108

蒙特卡洛树搜索

- 蒙特卡洛树搜索（Monte-Carlo Planning）
 - 单一状态蒙特卡洛规划：多臂赌博机 (multi-armed bandits)
 - 上限置信区间策略 (Upper Confidence Bound Strategies, UCB)
 - 蒙特卡洛树搜索 (Monte-Carlo Tree Search)
 - UCT (Upper Confidence Bounds on Trees)

109

蒙特卡洛树搜索



110

蒙特卡洛树搜索

• 单一状态蒙特卡洛规划：多臂赌博机（multi-armed bandits）

- 单一状态， k 种行动（即有 k 个摇臂）
- 在摇臂赌博机问题中，每次以随机采样形式采取一种行动 a ，好比随机拉动第 k 个赌博机的臂膀，得到 $R(s, a_k)$ 的回报。
- 问题：下一次需要拉动那个赌博机的臂膀，才能获得最大回报呢？



111

蒙特卡洛树搜索

• 单一状态蒙特卡洛规划：多臂赌博机（multi-armed bandits）

- 多臂赌博机问题是一种序列决策问题，这种问题需要在利用（exploitation）和探索（exploration）之间保持平衡。
- 利用（exploitation）：保证在过去决策中得到最佳回报
- 探索（exploration）：寄希望在未来能够得到更大回报

112

蒙特卡洛树搜索

• 单一状态蒙特卡洛规划：多臂赌博机（multi-armed bandits）

- 如果有 k 个赌博机，这 k 个赌博机产生的操作序列为 $X_{i,1}, X_{i,2}, \dots (i = 1, \dots, K)$ 。在时刻 $t = 1, 2, \dots$ ，选择第 i_t 个赌博机后，可得到奖赏 $X_{i_t,t}$ ，则在 n 次操作 $i_1, \dots, i_n$ 后，可如下定义悔值函数：

$$R_n = \max_{i=1, \dots, k} \sum_{t=1}^n X_{i,t} - \sum_{t=1}^n X_{i_t,t}$$

- 悔值函数表示了如下意思：在第 t 次对赌博机操作时，假设知道哪个赌博机能够给出最大奖赏（虽然在现实生活中这是不存在的），则将得到的最大奖赏减去实际操作第 i_t 个赌博机所得到的奖赏。将 n 次操作的差值累加起来，就是悔值函数的结果。
- 很显然，一个良好的多臂赌博机操作的策略是在不同人进行了多次玩法后，能够让悔值函数的方差最小。

113

蒙特卡洛树搜索

• 上限置信区间（Upper Confidence Bound, UCB）

- 在多臂赌博机的研究过程中，上限置信区间（Upper Confidence Bound, UCB）成为一种较为成功的策略学习方法，因为其在探索-利用（exploration-exploitation）之间取得平衡。
- 在UCB方法中，使 $X_{i,T_i(t-1)}$ 来记录第 i 个赌博机在过去 $t-1$ 时刻内的平均奖赏，则在第 t 时刻，选择使如下具有最佳上限置信区间的赌博机：

$$I_t = \max_{i \in \{1, \dots, k\}} \{ \bar{X}_{i,T_i(t-1)} + c_{t-1,T_i(t-1)} \}$$

其中 $c_{t,s}$ 取值定义如下：

$$c_{t,s} = \sqrt{\frac{2 \ln t}{s}}$$

$T_i(t) = \sum_{s=1}^t \prod(I_s = i)$ 为在过去时刻（初始时刻到 t 时刻）过程中选择第 i 个赌博机的次数总和。

114

蒙特卡洛树搜索

• 上限置信区间（Upper Confidence Bound, UCB）

也就是说，在第 t 时刻，UCB算法一般会选择具有如下最大值的第 j 个赌博机：

$$UCB = \bar{x}_j + \sqrt{\frac{2 \ln n}{n_j}} \text{ 或者 } UCB = \bar{x}_j + C \times \sqrt{\frac{2 \ln n}{n_j}}$$

$\bar{x}_j$ 是第 j 个赌博机在过去时间内所获得的平均奖赏值， n_j 是在过去时间内拉动第 j 个赌博机臂膀的总次数， n 是过去时间内拉动所有赌博机臂膀的总次数。 C 是一个平衡因子，其决定着在选择时偏重探索还是利用。

从这里可看出UCB算法如何在探索-利用（exploration-exploitation）之间寻找平衡：既需要拉动在过去时间内获得最大平均奖赏的赌博机，又希望去选择拉动臂膀次数最少的赌博机。

115

蒙特卡洛树搜索

• 上限置信区间（Upper Confidence Bound, UCB）

● UCB算法描述

Deterministic policy: UCB1.

Initialization: Play each machine once.

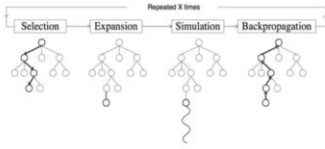
Loop:

- Play machine j that maximizes $\bar{x}_j + \sqrt{\frac{2 \ln n}{n_j}}$, where $\bar{x}_j$ is the average reward obtained from machine j , n_j is the number of times machine j has been played so far, and n is the overall number of plays done so far.

116

蒙特卡洛树搜索

- 蒙特卡洛树搜索 (Monte-Carlo Tree Search)
- 将上限置信区间算法UCB应用于游戏树的搜索方法，由Kocsis和Szepesvari在2006年提出
- 包括了四个步骤：选举(selection)，扩展(expansion)，模拟(simulation)，反向传播(Back-Propagation)



L. Kocsis and C. Szepesvari, Bandit based Monte-Carlo Planning, *ECML*, 2006:282–293

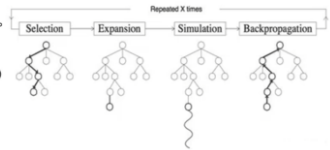
117

蒙特卡洛树搜索

- 蒙特卡洛树搜索 (Monte-Carlo Tree Search)

- 选择:
 - 从根节点 R 开始，向下递归选择子节点，直至选择一个叶子节点 L。
 - 具体来说，通常用UCB1 (Upper Confidence Bound, 上限置信区间) 选择最具“潜力”的后续节点

$$UCB = \bar{X}_j + \sqrt{\frac{2 \ln n}{n_j}}$$

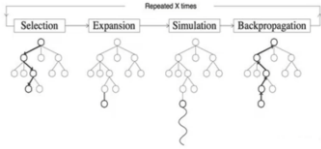


118

蒙特卡洛树搜索

- 蒙特卡洛树搜索 (Monte-Carlo Tree Search)

- 扩展:
 - 如果 L 不是一个终止节点（即博弈游戏不），则随机创建其后的一个未被访问节点，选择该节点作为后续子节点 C。

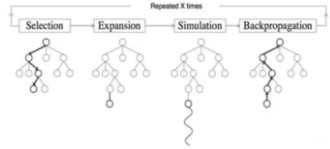


119

蒙特卡洛树搜索

- 蒙特卡洛树搜索 (Monte-Carlo Tree Search)

- 模拟:
 - 从节点 C 出发，对游戏进行模拟，直到博弈游戏结束。
- 反向传播
 - 用模拟所得结果来回溯更新导致这个结果的每个节点中获胜次数和访问次数。



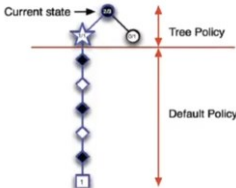
120

蒙特卡洛树搜索

- 蒙特卡洛树搜索 (Monte-Carlo Tree Search)

两种策略学习机制：

- 搜索树策略：从已有的搜索树中选择或创建一个叶子结点（即蒙特卡洛中选择和拓展两个步骤）。搜索树策略需要在利用和探索之间保持平衡。
- 模拟策略：从非叶子结点出发模拟游戏，得到游戏仿真结果。

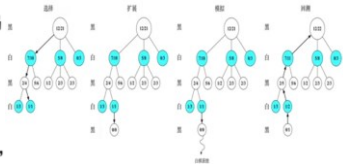


121

蒙特卡洛树搜索

- 蒙特卡洛树搜索：以围棋为例

- 以围棋为例，假设根节点是执黑棋方。
- 图中每一个节点都代表一个局面，每一个局面记录两个值 A/B:
 - A：该局面被访问中黑棋胜利次数。对于黑棋表示己方胜利次数，对于白棋表示己方失败次数（对方胜利次数）；
 - B：该局面被访问的总次数。



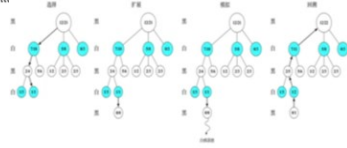
122

蒙特卡洛树搜索

• 蒙特卡洛树搜索：以围棋为例

该图刻画了蒙特卡洛树搜索四个步骤，假设根结点由黑棋行棋，为了选择根节点后续节点，需要由UCB1公式来计算根节点后续节点如下值，取一个值最大的节点为后续节点：

左1：7/10对应的局面奖赏值为 $\frac{7}{10} + \sqrt{\frac{\log(21)}{10}} = 1.252$
左2：5/8对应的局面奖赏值为 $\frac{5}{8} + \sqrt{\frac{\log(21)}{8}} = 1.243$
左3：0/3对应的局面评估分数为 $\frac{0}{3} + \sqrt{\frac{\log(21)}{3}} = 1.007$
由此可见，黑棋会选择局面7/10进行行棋。

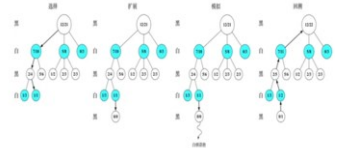


蒙特卡洛树搜索

• 蒙特卡洛树搜索：以围棋为例

在节点7/10，由白棋行棋，评估该节点
下面的两个局面，由UCB1公式可得（注意：此时A记录的是白棋失败的次数，所以第一项为1-A/B）：

左1，2/4对应的局面奖赏为 $1 - \frac{2}{4} + \sqrt{\frac{\log(21)}{4}} = 1.372$
左2，5/6对应的局面奖赏为 $1 - \frac{5}{6} + \sqrt{\frac{\log(21)}{6}} = 0.879$
由此可见，白棋会选择局面2/4进行行棋。

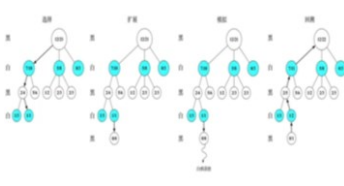


蒙特卡洛树搜索

• 蒙特卡洛树搜索：以围棋为例

在节点2/4，黑棋评估下面的两个局面，由UCB1公式可得：

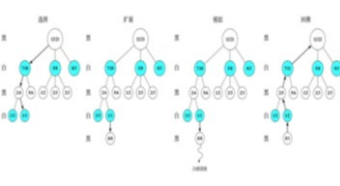
左1，1/3对应的局面奖赏为 $\frac{1}{3} + \sqrt{\frac{\log(21)}{3}} = 1.341$
左1，1/1对应的局面奖赏为 $\frac{1}{1} + \sqrt{\frac{\log(21)}{1}} = 2.745$
由此可见，黑棋会选择局面1/1进行行棋。此时已经到达叶子结点，需要进行扩展。



蒙特卡洛树搜索

• 蒙特卡洛树搜索：以围棋为例

随机扩展一个新节点。由于该新节点未被访问，所以初始化为0/0，接着在该节点下进行模拟。假设经过一系列仿真行棋后，最终白棋获胜。根据仿真结果来更新该仿真路径上每个节点的A/B值，该新节点的A/B值被更新为0/1，并向上回溯到该仿真路径上新节点的所有父节点，即所有父节点的A不变，B值加1。

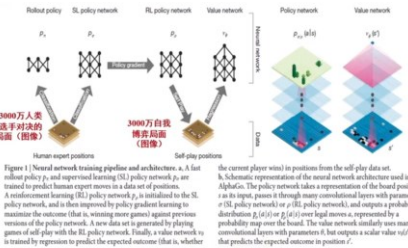


蒙特卡洛树搜索

• Monte-Carlo Tree Search (MCTS)：蒙特卡洛树搜索基于采样来得到结果、而非穷尽式枚举（虽然在枚举过程中也可剪掉若干不影响结果的分支）。

蒙特卡洛树搜索

• 蒙特卡洛树搜索应用：AlphaGo



- 将每个状态（局面）均视为一幅图像
- 训练策略（policy）网络和价值（value）网络
- $P_{\theta}(a|s)$ 表示当前状态为s（局面）时，采取行动a后所得到的概率； $v_{\theta}(s')$ 表示当前状态为s'时，整盘棋获胜的概率。

David Silver, et al., Mastering the game of Go with Deep Neural Networks and Tree Search, *Nature*, 529:484-490, 2016

蒙特卡洛树搜索

- 蒙特卡洛树搜索应用：AlphaGo：策略网络
- 基于监督学习来先训练策略网络
 - Idea: perform supervised learning (SL) to predict human moves
 - Given state s , predict probability distribution over moves a , $p_{\sigma,p}(a|s)$
 - Trained on 30M positions, 57% accuracy on predicting human moves
 - Also train a smaller, faster rollout policy network (24% accurate)
- 再基于强化学习来训练策略网络
 - Idea: fine-tune policy network using reinforcement learning (RL)
 - Initialize RL network to SL network
 - Play two snapshots of the network against each other, update parameters to maximize expected final outcome
 - RL network wins against SL network 80% of the time, wins against open-source Pachi Go program 85% of the time

129

蒙特卡洛树搜索

- 蒙特卡洛树搜索应用：AlphaGo：价值网络
- Value network
 - Idea: train network for position evaluation
 - Given state s' , estimate $v_{\theta}(s')$, expected outcome of play starting with position s and following the learned policy for both players
 - Train network by minimizing mean squared error between actual and predicted outcome
 - Trained on 30M positions sampled from different self-play games

130

蒙特卡洛树搜索

- 蒙特卡洛树搜索应用：AlphaGo
- 在通过深度学习得到的策略网络和价值网络帮助之下，如下完成棋局局面的选择和搜索。给定节点 v_0 ，将具有如下最大值的节点 v 选择作为 v_0 的后续节点
- $$\frac{Q(v)}{N(v)} + \frac{P(v|v_0)}{1 + N(v)}$$
- 这里 $P(v|v_0)$ 的值由策略网络计算得到。
 - 在模拟策略阶段(default policy), AlphaGo 不仅考虑仿真结果，而且考虑价值网络计算结果。
 - 策略网络和价值网络是离线训练得到的。

131

蒙特卡洛树搜索

- 蒙特卡洛树搜索应用：AlphaGo

Figure 3 Monte Carlo tree search in AlphaGo. a. Each simulation traverses the tree by selecting the edge with maximum action value Q , plus a bonus $u(P)$ that depends on a stored prior probability P for that edge. b. The leaf node may be expanded; the new node is processed once by the policy network p_{σ} and the output probabilities are stored as prior probabilities P for each action. c. At the end of a simulation, the leaf node is evaluated in two ways: using the value network v_{θ} and by running a rollout to the end of the game with the fast rollout policy p_{σ} , then computing the winner with function r . d. Action values Q are updated to track the mean value of all evaluations $r(\cdot)$ and $v_{\theta}(\cdot)$ in the subtree below that action.

价值网络计算

策略网络计算

$$a_t = \underset{a}{\operatorname{argmax}} (Q(s_t, a) + u(s_t, a))$$
$$N(s, a) = \sum_{i=1}^n 1(s, a, i)$$
$$Q(s, a) = \frac{1}{N(s, a)} \sum_{i=1}^n 1(s, a, i) V(r_i)$$
$$u(s, a) \propto \frac{P(s, a)}{1 + N(s, a)}$$

132

蒙特卡洛树搜索

- 蒙特卡洛树搜索应用：AlphaGo Zero

PenicillinLP bilibili

133